

Project Documentation: Learn4All

Inclusive Learning Progress & Resource Planner

 **Team Number:**

13

 **Theme:**

Python Fundamentals +
Object-Oriented
Programming (OOP) +
Sustainable Development
Goals (SDGs)

 **SDG Alignment:**

SDG 4 (Quality Education)
& SDG 10 (Reduced
Inequalities)

1. Detailed Problem Statement and Research

Many primary and secondary students (Classes 1–10) face unequal access to tailored learning support. Traditional learning systems train each student equally, leaving behind students with specific learning gaps or distinct accessibility needs.

-  **Educational Inequality:** Students struggling with specific topics lack structured feedback to identify exact weaknesses.
-  **Accessibility Gaps:** Existing tools rarely cater to differing resource preferences (e.g., text-only materials for low bandwidth/screen readers vs. captioned videos for hearing-impaired learners).

 Learn4All addresses this problem by acting as an automated diagnostic planner that maps individual performance, flags weak topics, and matches students with targeted, accessible study materials.

2. Project Objectives and Proposed Solution

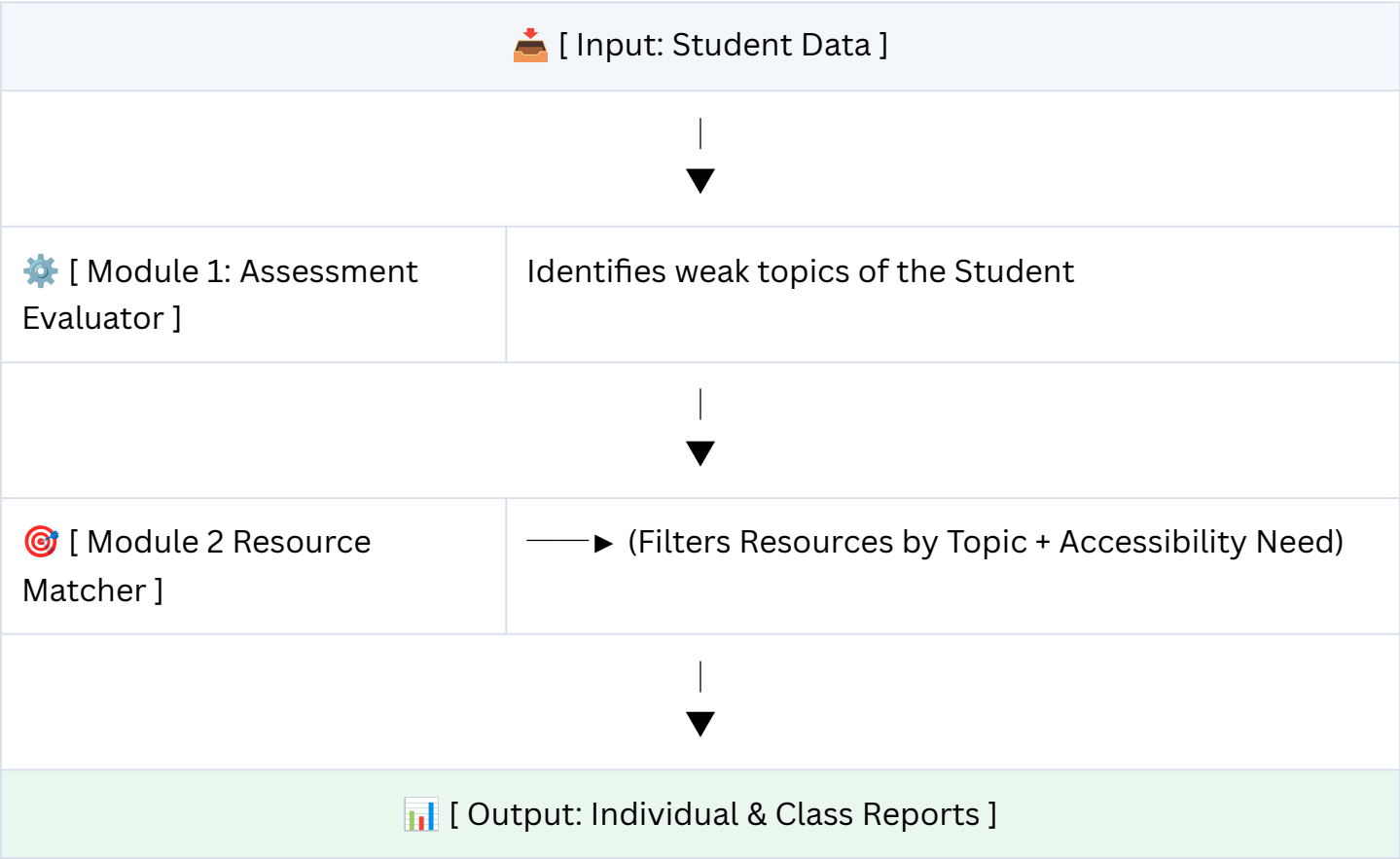
- 📌 **Core Objective:** Build a modular Python application utilizing OOP principles to track student performance and recommend study paths based on learning gaps and accessibility requirements.

🔧 **Proposed Solution:**

- **Score Assessment:** Evaluate student assessment scores per subject/topic.
- **Weakness Detection:** Dynamically flag weak topics where student performance falls below 60%.
- **Accessibility Matching:** Filter learning resources matching the weak topic while strictly enforcing accessibility compatibility (Text-Only, Captioned Video, Audio).
- **Report Generation:** Generate both individual learner reports and class-level summary reports.

3. 🔄 System Workflow / Flowchart

The system operates through a sequential 4-stage pipeline:



4. Detailed Explanation of How the Project Works

Object-Oriented Programming (OOP) Implementation

The project implements four core domain classes and one controller class:

- **Assessment:** Captures assessment details (subject, topic, `max_marks`, `scored_marks`). Computes percentage scores using `get_percentage()`.
 - **Resource:** Stores educational material details (`resource_id`, title, `res_type`, topic, `accessibility_tag`).
 - **Student:** Encapsulates learner profile (`student_id`, name, `accessibility_need`) and assessment history. Features `get_performance_summary()` to categorize topics into strengths ($\geq 60\%$) and weaknesses ($< 60\%$).
 - **LearningPlannerEngine:** Serves as the system controller managing repository lists for students and resources. It implements `recommend_resources()` to match student weak topics with accessibility-compatible resources.
-

5. Non-Applicable Sections (Hardware / AI/ML)

-  **AI/ML Model Information:** N/A. The system deliberately utilizes deterministic logic algorithms (threshold processing and relational filtering) to ensure fast, reliable performance without heavy dependencies.
 -  **Hardware Components & Circuit Info:** N/A. Learn4All is a pure software desktop/terminal application.
-

6. Testing and Results

Testing was conducted using edge cases across different student accessibility profiles:

-  **Test Case 1 (Text-Only Preference):** Student scoring 45% in OOP Concepts received OOP Basics (Text Guide).

-  **Test Case 2 (Captioned Video Preference):** Student scoring 40% in Loops received Python Loops Walkthrough.

 **Class Summary Verification:** Aggregated weakness counts accurately identified class-wide trouble spots to support teacher interventions.

7. Challenges Faced and Solutions

 **Challenge:** Difficulty because of wide range of classes and topics

 **Solution:** Used AI for repetitive tasks

 **Challenge:** Ensuring exact resource matching without recommending materials the student cannot access.

 **Solution:** Applied multi-condition filtering matching both topic AND `accessibility_tag` parameters.

8. Future Scope

-  **Database Integration:** Connecting SQLite/PostgreSQL for persistent data storage.
-  **Web Dashboard:** Expansion using Streamlit or Flask for browser-based teacher control.
-  **Gamified Learning:** Dynamic quiz creation based on difficulty levels and badges.

9. Team contribution

Aarav-Researched on the problem, created presentation, code and documentation

Pakhi- Very little support

Chetan- Very little support